AI-Based Bus Crowd Prediction System using Machine Learning

Problem Statement:

Public transportation is an important part of daily life, especially for students and working professionals. However, passengers often face overcrowded buses during peak hours, examination periods, and busy travel times, which makes it difficult to choose a suitable bus and plan a journey comfortably.

Manual estimation of bus crowd levels is not always reliable. Therefore, an automated system is needed to predict the expected passenger count and crowd level based on bus details, travel date, travel time, and available historical route data. This project develops a web-based RealBusCrowd - AI-Based Bus Crowd Prediction System using Python, Flask, and Machine Learning. The system allows users to search for a bus or route, select a bus number, enter travel details, and obtain an estimated passenger count, occupancy percentage, and crowd classification.

Objectives:

1. To develop a web-based bus crowd prediction system.
2. To collect and process bus route and crowd-related datasets.
3. To apply Machine Learning techniques for predicting bus crowd levels.
4. To provide a user-friendly interface for selecting bus numbers and travel details.
5. To estimate passenger occupancy based on the predicted passenger count and bus capacity.
6. To classify crowd levels as Low, Medium, or High.
7. To help passengers plan their journeys by providing estimated crowd information.

Software / Hardware Required:

Software Requirements

• Operating System: Windows

• IDE: Visual Studio Code

• Programming Language: Python 3.x

• Web Framework: Flask

• Machine Learning: scikit-learn

• Data Processing: Pandas, NumPy

• Frontend: HTML5, CSS3

• Dataset Format: CSV

Hardware Requirements

• Computer or laptop

• Minimum 4 GB RAM

• Processor: Intel i3 or equivalent

• Internet connection for development and accessing the web application

Project Directory Structure:

The project ("RealBusCrowd") is organized as a Flask-based web application with separate frontend, styling, prediction logic, and dataset files, as shown below.

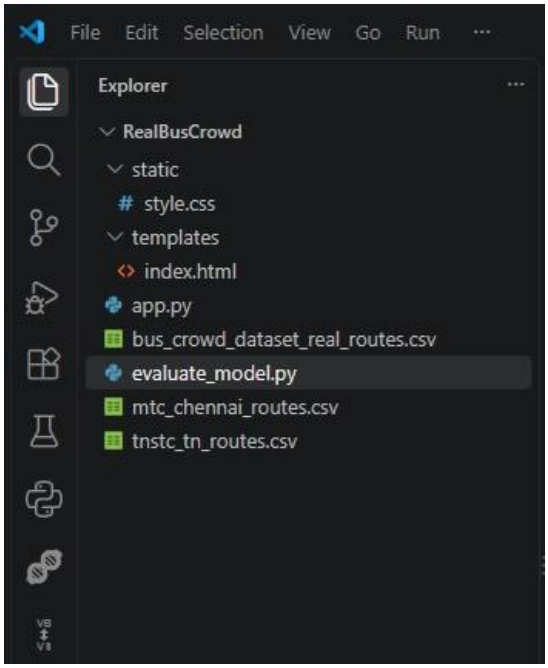


Fig 1: Project directory structure of RealBusCrowd in Visual Studio Code

Procedure:

1. Create the project directory and configure the Python development environment.

2. Collect and organize bus route and crowd-related information in CSV files (bus_crowd_dataset_real_routes.csv, mtc_chennai_routes.csv, tnstc_tn_routes.csv).

3. Load the dataset using Pandas and preprocess the available data.

4. Identify relevant input features for crowd prediction, such as bus route, travel time, weather, and exam-period information, depending on dataset availability.

5. Train and evaluate a suitable Machine Learning model using the prepared dataset.

6. Develop the Flask backend in app.py to handle user inputs and prediction requests.

7. Create the web interface using HTML and CSS.

8. Allow users to search for a bus or route and select the required bus number.

9. Enter the travel date and time and submit the details using the Predict Crowd button.

10. Generate the estimated passenger count and calculate the occupancy percentage using the bus capacity.

11. Display the predicted crowd level and relevant travel information on the results page.

12. Evaluate the model using evaluate_model.py and report the metrics obtained from the actual evaluation.

Implementation Code:

Representative implementation of the key modules used in the project, based on the pipeline shown in the application.

1. Flask Application (app.py)

```
from flask import Flask, render_template, request

app = Flask(_name_)

@app.route("/")
def home():
    return render_template("index.html")

@app.route("/predict", methods=["POST"])
def predict():
    bus_number  = request.form.get("bus_number")
    travel_date = request.form.get("travel_date")
    travel_time = request.form.get("travel_time")

    # Trained ML model generates the passenger estimate here
    passengers = predict_passengers(bus_number, travel_date, travel_time)
    capacity  = get_bus_capacity(bus_number)

    occupancy = calculate_occupancy(passengers, capacity)
    crowd     = classify_crowd(occupancy)

    return render_template("result.html",
        bus_number=bus_number, travel_date=travel_date,
        travel_time=travel_time, passengers=passengers,
        capacity=capacity, occupancy=occupancy, crowd=crowd)

if _name___== "_main_":
    app.run(debug=True)
```

2. Occupancy Percentage Calculation

```
def calculate_occupancy(passengers, capacity):
    if capacity <= 0:
        return 0
    occupancy = (passengers / capacity) * 100
    return round(occupancy, 1)
```

3. Crowd Level Classification

```
def classify_crowd(occupancy):
    if occupancy < 40:
        return "LOW CROWD"
    elif occupancy < 75:
        return "MEDIUM CROWD"
    else:
        return "HIGH CROWD"
```

4. Model Evaluation (evaluate_model.py)

```
from sklearn.metrics import (accuracy_score,
                             classification_report,
                             confusion_matrix)

# y_test: actual test labels, y_pred: model predictions
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Classification Report:")
print(classification_report(y_test, y_pred))
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

Output (Step-by-Step Screenshots):

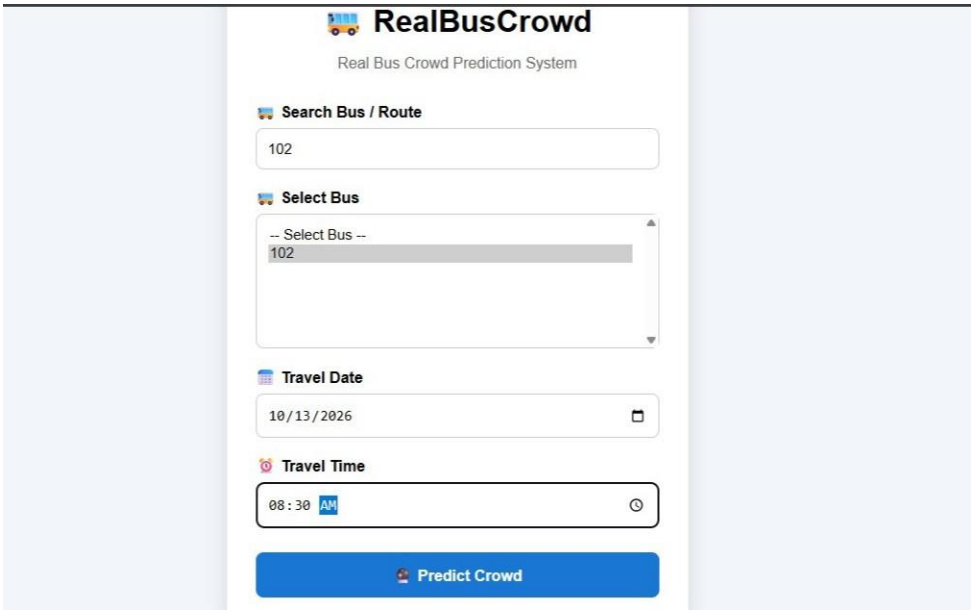


Fig 2: Step 1 — Home page: search bus/route (102), select bus, enter travel date (13-10-2026) and travel time (08:30 AM), then click Predict Crowd

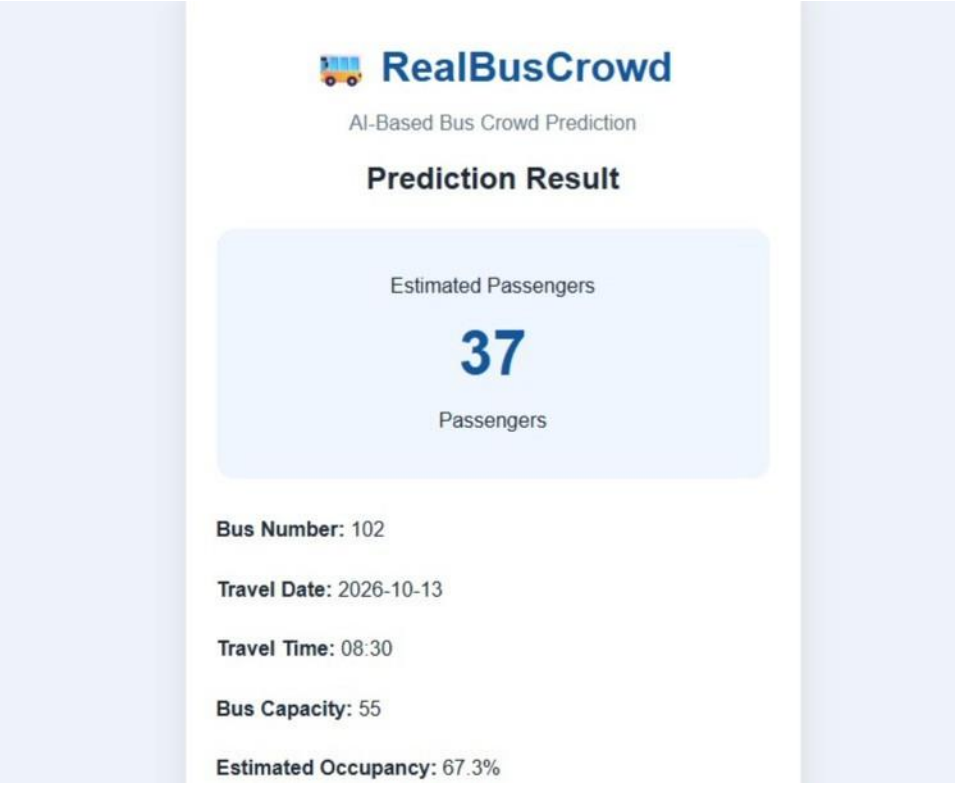


Fig 3: Step 2 — Prediction result: estimated passengers (37) for bus 102 with travel date, time, bus capacity (55) and estimated occupancy (67.3%)

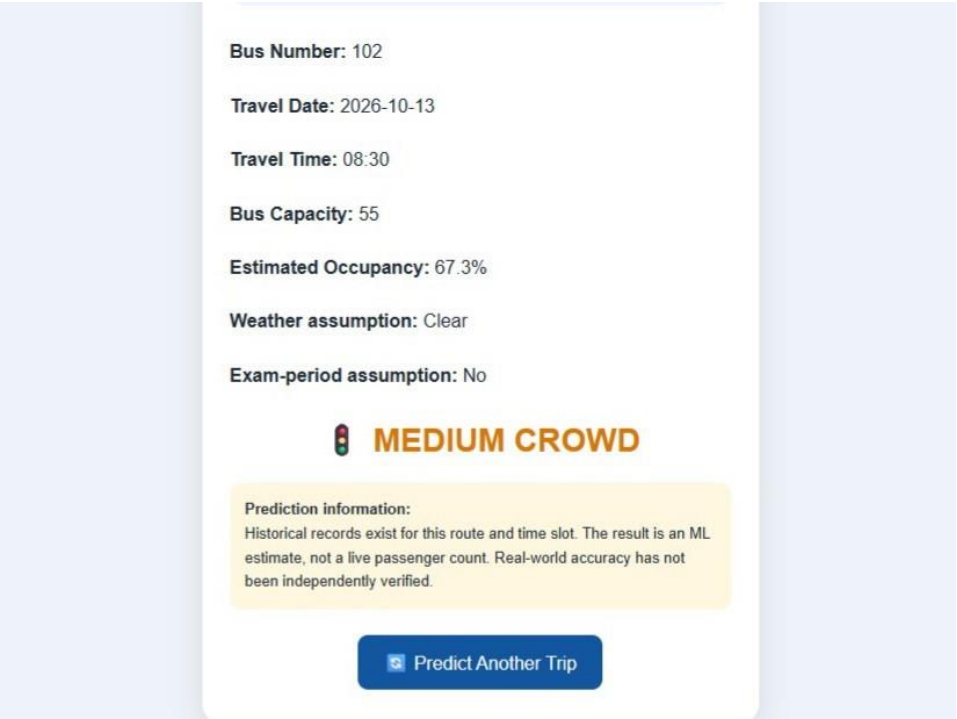


Fig 4: Step 3 — Predicted crowd level: MEDIUM CROWD, with weather assumption (Clear), exam-period assumption (No) and prediction information note

Result:

The RealBusCrowd - AI-Based Bus Crowd Prediction System was successfully developed as a web application using Python, Flask, HTML, CSS, and Machine Learning components. The application provides an interface for selecting a bus or route, entering the travel date and time, and viewing an estimated passenger count and crowd level.

For bus 102 on 13-10-2026 at 08:30, the system estimated 37 passengers on a bus of capacity 55, giving an estimated occupancy of 67.3%, which was classified as MEDIUM CROWD**.** The result page also states the weather and exam-period assumptions used, and notes that the output is an ML estimate and not a live passenger count.

The project demonstrates how a web-based Machine Learning application can support bus travel planning through estimated crowd prediction. Further improvements can include verified historical passenger data, route-specific training, and validation against actual passenger counts.